

Tailscale — DERP relay avoidance

Who's actually on the tailnet

Only **15 nodes** run the Tailscale client: the 14 GL-iNet A-1300 routers (12 theatres + 2 VMix nodes) and the mothership's Tailscale container. Each advertises its own subnet (`--advertise-routes`) and accepts the others' (`--accept-routes`) — see `config/tailscale-up-all-devices.sh` .

No individual client device runs Tailscale. Not the PowerPoint/VT laptops, not the control laptops, not the ATEM, not BirdDog Play/Central, not the VMix PCs or P400 cameras. They all reach the rest of the network purely through their GL-iNet's normal LAN gateway — subnet routing makes `192.168.1.0/24` (and every other theatre's subnet, where ACLs allow it) reachable transparently, with zero Tailscale software installed on any of them.

Why direct connections are likely

Every node ultimately sits behind one shared mothership internet connection — a favorable case for direct (non-relayed) connections, since Tailscale can either use the true private LAN path or hairpin through the single shared public IP.

What determines it in practice:

1. **Whether the Ubiquiti firewall allows UDP between VLANs/subnets.** If yes → direct private-path connections, no internet round-trip.
2. **If inter-VLAN UDP is blocked**, Tailscale falls back to NAT hairpin via the shared public IP — usually works, but more fragile with GL-iNet's own NAT stacked underneath (double-NAT).
3. **DERP relay** is only the fallback when both of the above fail. Tailscale continuously retries and upgrades to direct when possible.

Verifying after setup

```
tailscale status      # shows direct vs relay per peer
tailscale ping <peer>
```

Self-hosted DERP server (decided — running on the mothership)

Runs as a Docker container on the consolidated Unraid server — `derp-server` , `192.168.1.16` (see `docs/ip-address-map.md`) — using Tailscale's own `derper` binary/image (official docs).

Why this still needs a real, reachable hostname. DERP isn't reached over an already-established Tailscale tunnel to the peer that needs relaying — a node holds a persistent connection to its DERP server over its own normal WAN path, independent of any specific peer, precisely so it can bootstrap connectivity when direct WireGuard isn't available yet. That means the venue's shared-internet-connection hairpin trick (see above) is exactly what makes this work here: every theatre's GL-iNet reaches the DERP server's hostname over its *own* WAN path, which loops back through the same shared public IP without the traffic ever actually leaving the venue's router. This requires:

- A real DNS hostname pointed at the mothership's public IP (a domain you control, or a dynamic-DNS hostname if you don't have one). Named `derp.example.net` — needs to actually be registered/pointed (or swapped for whatever domain/DDNS host you do control) before it resolves to anything.
- A port forward on the Cloud Gateway: WAN `443/tcp` → `192.168.1.16:443`, and WAN `3478/udp` → `192.168.1.16:3478` (STUN). See `config/unifi/network-config.yaml` . **443** for the DERP listener — matches Tailscale's own DERP fleet and is the most firewall-friendly port (many restrictive networks allow outbound 443 and nothing else), though it doesn't matter much for reachability here specifically since it's an internal hairpin, not roaming clients behind a hotel firewall.
- Let's Encrypt issues the cert automatically against that hostname (`-certmode=letsencrypt`) since the port forward makes it genuinely reachable for the ACME challenge.

```
docker run -d --name derp-server \
  --network macvlan-mothership --ip 192.168.1.16 \
  -v /mnt/user/appdata/derper/certs:/app/certs \
  -v /var/run/tailscale/tailscaled.sock:/var/run/tailscale/tailscaled.sock \
  tailscale/derper \
  -hostname=derp.example.net \
  -certmode=letsencrypt -certdir=/app/certs \
  -stun -stun-port=3478 \
  -verify-clients
```

(Attached to the `macvlan-mothership` network at `192.168.1.16` — the same macvlan setup every other mothership container uses, so no `-p` port publishes, which are inert under macvlan anyway. The canonical definition of this service lives in `config/docker-compose.yml`; this standalone command is just the same thing spelled out.)

`-verify-clients` restricts relay use to nodes already on this tailnet — needs the container to reach the host's `tailscaled`, which is exactly what the `tailscaled.sock` mount above is for. Recommended so the DERP server isn't an open relay; drop both the flag and the mount if that's not a concern.

Register it with the tailnet by adding a custom region to the ACL policy — see the `derpMap` block in `config/tailscale-acl.json`. Kept `OmitDefaultRegions: false` (the safer default) — Tailscale prefers this DERP node when it's faster, but still falls back to the public regions if the self-hosted one goes down, rather than losing relay fallback entirely.

Verify with `tailscale netcheck` — it reports which DERP region a node prefers.

Config sketch

See `config/tailscale-up-commands.sh` for the per-role `tailscale up` invocations and `config/tailscale-acl.json` for the ACL policy.

Routes still need approving in the Tailscale admin console (or via `autoApprovers` keyed to tag/CIDR) before they take effect — advertising a route alone is not sufficient.